

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.023

ОТЧЕТ

по исследовательскому курсовому проекту

на тему

**«Оптимизация алгоритмов маршрутизации на примере задачи
нескольких коммивояжеров»**

по направлению подготовки бакалавров 09.03.04 «Программная инженерия»

Выполнил
студент группы БПИ246
Д. Д. Купцов

Научный руководитель
Ермаков Андрей Максимович

Москва 2026

РЕФЕРАТ

Отчет содержит описание исследовательского курсового проекта, посвященного задаче множественного коммивояжера и практическому сравнению эвристических алгоритмов маршрутизации. В работе рассматривается постановка mTSP с одним депо и целевой функцией MINSUM, при которой требуется минимизировать суммарную длину всех маршрутов. Объектом исследования являются процессы маршрутизации и распределения точек между несколькими агентами, предметом исследования являются эвристические и метаэвристические методы решения mTSP при ограниченном времени вычислений.

Цель работы --- разработать и экспериментально оценить масштабируемые эвристические методы оптимизации маршрутов для задачи множественного коммивояжера. Для достижения цели были изучены классические подходы к TSP и mTSP, реализованы базовые алгоритмы и семейство LKH-подобных оберток, подготовлен экспериментальный контур на C++ и Python, проведены сравнения с OR-Tools, преобразованием TSP $\rightarrow$ mTSP и внешним решателем LKH-3. Полученные результаты показывают, что LKH-подобные методы существенно превосходят простые конструктивные эвристики по качеству маршрутов, однако при сравнении с полноформатным LKH-3 возникает компромисс: собственные быстрые версии работают заметно быстрее, но обычно уступают по значению целевой функции.

Ключевые слова: задача коммивояжера, множественный коммивояжер, mTSP, маршрутизация, эвристические алгоритмы, Lin--Kernighan--Helsgaun, локальный поиск, MINSUM, вычислительный эксперимент.

Нужно добавить перед сдачей. После финальной правки нужно обновить строку с объемом работы: количество страниц, таблиц, рисунков, источников и приложений. Это лучше сделать после окончательной компиляции PDF.

Содержание

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	5
ВВЕДЕНИЕ	7
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	10
1.1 Классическая задача коммивояжера	10
1.2 Задача множественного коммивояжера	10
1.3 Эвристические и метаэвристические подходы	11
1.4 Масштабирование и candidate sets	12
2 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ	13
2.1 Формальная постановка в проекте	13
2.2 Экспериментальный контур	13
2.3 Набор реализованных алгоритмов	14
2.4 Особенности LKH-подобной реализации	15
3 МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА	16
3.1 Семейства тестовых инстансов	16
3.2 Метрики	16
3.3 Валидация результатов	17
4 РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ	17
4.1 Базовое сравнение на равномерных инстансах	17
4.2 Сравнение с OR-Tools	18
4.3 Расширенный benchmark по семействам инстансов	19
4.4 Эволюция LKH-подобных версий	20
4.5 Крупные инстансы и проблема масштабирования	20
4.6 Сравнение v12 с внешним LKH-3 baseline	21
4.7 Интерпретация результатов	22
5 ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ	23
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ПРИЛОЖЕНИЕ А. ЧТО НУЖНО ДОБАВИТЬ В ФИНАЛЬНУЮ ВЕРСИЮ ПЕРЕД СДАЧЕЙ	27
ПРИЛОЖЕНИЕ Б. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБ- ЛИЦЫ ЭКСПЕРИМЕНТОВ	28

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Термин	Определение
TSP	Travelling Salesman Problem, задача коммивояжера: требуется построить кратчайший замкнутый маршрут, проходящий через все заданные вершины ровно один раз.
mTSP	Multiple Travelling Salesman Problem, задача множественного коммивояжера: требуется построить несколько маршрутов, начинающихся и заканчивающихся в общем депо, так чтобы каждая клиентская вершина была посещена ровно одним маршрутом.
Депо	Специальная вершина, из которой начинаются и в которой завершаются маршруты всех агентов.
Агент, коммивояжер	Исполнитель, которому назначается отдельный маршрут в решении mTSP.
MINSUM	Целевая функция mTSP, равная суммарной длине всех маршрутов. В данной работе меньшая MINSUM означает более качественное решение.
MINMAX	Альтернативная целевая функция, в которой минимизируется длина самого длинного маршрута. В работе она используется как вспомогательный контекст, но не как основной критерий.
2-opt	Локальное улучшение маршрута, при котором удаляются два ребра и фрагмент маршрута разворачивается, если это уменьшает длину.
k-opt	Обобщение 2-opt: из маршрута удаляется k ребер, после чего фрагменты соединяются новым способом.
LKH	Lin--Kernighan--Helsgaun, семейство высокоэффективных эвристик для TSP, основанных на переменной глубине локального поиска и кандидатных ребрах.

Термин	Определение
Candidate set	Ограниченный набор перспективных соседей вершины, используемый для ускорения локального поиска и отказа от полного перебора всех пар вершин.
GRASP	Greedy Randomized Adaptive Search Procedure, жадно-рандомизированная процедура построения и улучшения решений.
OR-Tools	Библиотека Google для решения задач оптимизации, в работе используется как внешний эвристический ориентир для маршрутизации.
FILLO2	Эвристический подход для крупномасштабных задач маршрутизации, используемый как внешний ориентир по идеям масштабирования.
Benchmark	Набор тестовых экземпляров задачи, на котором сравниваются алгоритмы.
Relative gap	Относительное отклонение решения от выбранного ориентира: $100 \cdot (A - B)/B$, где A --- значение исследуемого алгоритма, B --- значение базового решения.

ВВЕДЕНИЕ

Задачи маршрутизации относятся к числу наиболее практически значимых задач комбинаторной оптимизации. Они возникают в логистике, транспортном планировании, управлении цепями поставок, распределении сервисных бригад, проектировании производственных процессов и других областях, где требуется упорядочить перемещение между множеством точек. Классическим математическим ядром таких задач является задача коммивояжера, в которой необходимо найти маршрут минимальной длины, проходящий через все вершины ровно один раз и возвращающийся в исходную вершину [1,2].

Несмотря на простую формулировку, TSP относится к NP-трудным задачам. Это означает, что при росте числа вершин полный перебор решений становится неприемлемым уже на сравнительно малых размерах. В прикладных постановках ситуация обычно еще сложнее: маршрут строится не для одного исполнителя, а для нескольких агентов, которые стартуют из общего депо и должны совместно обслужить набор клиентов. Такая постановка называется задачей множественного коммивояжера, или mTSP. В отличие от TSP, здесь требуется не только определить порядок обхода вершин, но и распределить вершины между маршрутами [4].

В данной работе рассматривается вариант mTSP с одним депо и целевой функцией MINSUM. Для практических задач такая постановка естественна: если несколько машин или исполнителей обслуживают один и тот же набор точек, то суммарная длина маршрутов может быть связана с расходом топлива, временем работы, стоимостью обслуживания или суммарной нагрузкой на систему. При этом получение точного оптимума для задач средней и большой размерности обычно не является реалистичной целью. Более важным становится поиск достаточно качественного приближенного решения в заранее заданный вычислительный бюджет.

Актуальность исследования определяется двумя обстоятельствами. Во-первых, mTSP является базовой моделью для более сложных задач маршрутизации, включая варианты Vehicle Routing Problem. Во-вторых, современные эвристические методы показывают, что сильное качество решения часто достигается не за счет полного перебора, а за счет аккуратного ограничения области поиска: кандидатных ребер, локальных перестановок, кластеризации, разреженных соседств и процедур повторного улучшения [3,7,8,10]. Поэтому практический

интерес представляет не только выбор известного решателя, но и анализ того, какие инженерные решения делают алгоритм масштабируемым.

Объектом исследования являются процессы маршрутизации и распределения маршрутов между несколькими агентами в задачах комбинаторной оптимизации. Предметом исследования являются эвристические и метаэвристические алгоритмы решения mTSP, а также их влияние на качество маршрутов и время вычислений при росте размерности задачи.

Цель работы --- разработать и проанализировать эффективные эвристические методы оптимизации маршрутов для задачи множественного коммивояжера. Для достижения цели в работе решаются следующие задачи:

1. изучить постановку TSP и mTSP, основные целевые функции и ограничения;
2. проанализировать существующие эвристические подходы, включая локальный поиск, GRASP, LKH и методы построения candidate sets;
3. реализовать единый экспериментальный контур для запуска алгоритмов и проверки корректности решений;
4. подготовить несколько семейств синтетических mTSP-инстансов, отражающих разные геометрические сценарии;
5. сравнить базовые эвристики, собственные LKH-подобные версии и внешние ориентиры по качеству, времени и валидности решений;
6. сформулировать выводы о применимости разработанных подходов и ограничениях текущей реализации.

В качестве рабочей гипотезы принимается следующее положение: эвристики, использующие ограниченные множества кандидатных ребер, локальный поиск и LKH-подобные процедуры улучшения, позволяют получать более качественные решения mTSP по MINSUM, чем простые конструктивные алгоритмы, при сопоставимых или приемлемых вычислительных затратах. Дополнительно проверяется более осторожная инженерная гипотеза: быстрые собственные версии могут быть полезны при жестком временном бюджете, даже если по качеству они уступают полноформатному внешнему LKH-3.

Методологическая основа работы включает анализ научной литературы, математическую формализацию задачи, программную реализацию алгоритмов, вычислительный эксперимент, валидацию маршрутов и сравнительный анализ результатов. Основные эксперименты проводятся на синтетических

инстансах с разными распределениями точек: равномерным, кластерным, кластерным со смещенным депо, смешанным с выбросами и стрессовыми сценариями с большим числом агентов.

Нужно добавить перед сдачей. Перед сдачей желательно добавить точную конфигурацию компьютера: процессор, объем RAM, ОС, компилятор, режим сборки Release/Debug и наличие OpenMP. В исходных материалах есть результаты запусков, но нет полного описания аппаратной среды, а для экспериментальной главы это важная деталь.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Классическая задача коммивояжера

Задача коммивояжера формулируется следующим образом. Пусть задано множество вершин $V = \{0, 1, \dots, n - 1\}$ и функция расстояния $d(i, j)$ между каждой парой вершин. Требуется найти перестановку вершин, задающую замкнутый маршрут минимальной длины:

$$\min_{\pi} \left(d(\pi_n, \pi_1) + \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) \right). \quad (1)$$

В евклидовой постановке вершины задаются координатами на плоскости, а расстояние вычисляется как длина отрезка между точками. Такая постановка удобна для экспериментов, поскольку позволяет генерировать тестовые данные контролируемым образом и сохраняет связь с прикладными задачами маршрутизации.

Точные методы решения TSP имеют большую теоретическую ценность, но для крупных экземпляров используются ограниченно. В ранних работах были предложены процедуры ветвей и границ, динамического программирования и другие точные подходы, однако рост пространства решений делает их дорогостоящими [1]. Поэтому в практических задачах широко применяются эвристики. Уже классический алгоритм Lin--Kernighan показал, что локальный поиск переменной глубины способен давать оптимальные или близкие к оптимальным решения на широком классе тестов [1]. Позднее Helsgaun развил этот подход в LKH, усилив его использованием кандидатных ребер и более сложной стратегии поиска [2].

1.2 Задача множественного коммивояжера

mTSP является естественным обобщением TSP. Пусть задано m агентов и общее депо 0. Решение представляет собой набор маршрутов:

$$R_s = (0, v_1^s, v_2^s, \dots, v_{k_s}^s, 0), \quad s = 1, \dots, m, \quad (2)$$

где каждая вершина из $V \setminus \{0\}$ встречается ровно в одном маршруте. В варианте MINSUM целевая функция имеет вид:

$$F_{\text{sum}}(R_1, \dots, R_m) = \sum_{s=1}^m \sum_{i=0}^{k_s} d(v_i^s, v_{i+1}^s), \quad v_0^s = v_{k_s+1}^s = 0. \quad (3)$$

В литературе также часто рассматривается MINMAX, где минимизируется длина самого длинного маршрута. Эти цели отражают разные практические смыслы. MINSUM ориентирован на суммарную стоимость обслуживания, а MINMAX --- на балансировку нагрузки между агентами. В данной работе основной критерий --- MINSUM, но баланс маршрутов дополнительно анализируется через отношение длины самого длинного маршрута к средней длине маршрута.

Обзор Bektaş показывает, что mTSP имеет несколько эквивалентных и близких формулировок, а также может решаться через преобразования к TSP, целочисленное программирование и эвристические методы [4]. Для задач большой размерности особенно важны эвристики, поскольку даже если точный метод может решить малый пример, он не дает приемлемого времени на крупных данных.

1.3 Эвристические и метаэвристические подходы

Простейший класс методов составляют конструктивные эвристики. Они строят решение последовательно, выбирая следующую вершину по локальному правилу, например по ближайшему соседу. Такие методы очень быстры, но часто принимают ранние решения, которые затем трудно исправить. В проекте к этой группе относится алгоритм rand+nn: он распределяет вершины между маршрутами с элементом случайности и ближайшего соседа.

Второй класс методов основан на локальном поиске. Для каждого маршрута можно применять 2-opt: если замена двух ребер уменьшает длину маршрута, соответствующий фрагмент разворачивается. В проекте этот подход представлен алгоритмом 2opt+greed. Он сильнее чистого ближайшего соседа, но все еще ограничен, потому что улучшает уже построенные маршруты и не всегда хорошо перераспределяет вершины между агентами.

Третий класс составляют рандомизированные многостартовые процедуры. GRASP строит несколько решений с использованием ограниченного спис-

ка лучших локальных кандидатов, затем улучшает каждое решение и выбирает лучшее [9]. В реализации проекта `grasp` использует `randomized restricted candidate list`, внутримаршрутный 2-opt и межмаршрутные обмены. Такой подход заметно повышает качество, но его время растет с числом итераций и размером инстанса.

Отдельное место занимают LKH-подобные методы. Их сила состоит в том, что поиск не рассматривает все возможные ребра, а концентрируется на малом наборе перспективных соседей. Для TSP эта идея поддержана классическими работами Lin--Kernighan и Helsgaun [1,2]. Для mTSP LKH-3 использует преобразование задач маршрутизации к стандартному симметричному TSP и штрафные функции для ограничений [3]. В частности, для mTSP с одним депо возможно добавить копии депо и получить структуру, распадающуюся на несколько маршрутов.

1.4 Масштабирование и candidate sets

Ключевая инженерная проблема крупных TSP/mTSP-инстансов --- невозможность рассматривать полный граф. Если для каждой вершины проверять все остальные вершины, построение соседств требует порядка $O(n^2)$ сравнений. Для $n = 100000$ это уже непрактично. Поэтому современные эвристики строят разреженный граф кандидатов.

В работах Taillard и Helsgaun по POPMUSIC показано, что качественные candidate sets можно строить с эмпирической сложностью ниже квадратичной даже для очень крупных TSP-инстансов [7]. Похожую идею используют крупномасштабные VRP-решатели: они ограничивают локальный поиск гранулярными соседствами, динамическими областями изменений и кэшами затронутых вершин [8]. Для данной работы это важно не как прямое заимствование готового алгоритма, а как подтверждение общего принципа: масштабируемость достигается через сужение пространства ходов и отказ от полного перебора.

В собственных версиях `lkh-wrapper` этот принцип реализован через геометрические candidate sets, пространственную сетку, кэширование расстояний, ограниченный 2-opt и time-budget. В более поздних версиях добавлены кластерные начальные маршруты, процедуры перераспределения блоков между маршрутами и отдельные бюджеты на построение первого решения и последующее улучшение.

2 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ

2.1 Формальная постановка в проекте

Входной файл mTSP-инстанса содержит число вершин n , число агентов m и координаты всех вершин. Вершина с индексом 0 считается депо. Требуется построить m маршрутов, удовлетворяющих условиям:

1. каждый маршрут начинается в депо и заканчивается в депо;
2. каждая клиентская вершина $1, \dots, n - 1$ встречается ровно в одном маршруте;
3. значение MINSUM минимизируется или, для приближенного алгоритма, становится как можно меньше при заданном бюджете времени.

Валидность решения проверяется отдельно от вычисления целевой функции. Это принципиально важно: эвристика может вернуть численно короткие маршруты, но если какая-либо вершина пропущена или посещена дважды, такое решение нельзя учитывать в сравнении.

2.2 Экспериментальный контур

Репозиторий проекта содержит C++-реализации решателей и Python-скрипты для генерации данных, пакетного запуска и построения отчетов. Основные элементы контура:

- `run_mtsp.py` --- единая точка запуска mTSP-решателей;
- `experiments/generate_mtsp_instances.py` --- генерация синтетических инстансов;
- `experiments/run_benchmarks.py` --- пакетный запуск алгоритмов по конфигурации;
- `python/mtsp_baselines/ortools_routing.py` --- внешний baseline на OR-Tools;
- `python/mtsp_baselines/tsp_transform_lkh.py` --- baseline через построение TSP-тура и последующее разбиение на маршруты;
- `data/results/` --- CSV и JSON-артефакты экспериментов.

Генерация инстансов → запуск решателей → проверка валидности маршрутов → пересчет MINSUM → агрегация CSV/JSON → сравнительный анализ
--

Рис. 1: Схема экспериментального контура

Такое разделение уменьшает риск смещения кода алгоритма и кода оценки. Решатель отвечает за построение маршрутов, а отдельная часть проекта проверяет корректность результата и пересчитывает objective по сохраненным маршрутам.

2.3 Набор реализованных алгоритмов

Алгоритм	Роль в исследовании
rand+nn	Быстрый конструктивный baseline: случайность и ближайший сосед. Нужен как нижняя граница качества для простых решений.
2opt+greed	Жадное построение с локальным 2-opt. Позволяет оценить вклад внутримаршрутного локального поиска.
grasp	Рандомизированный многостартовый подход с restricted candidate list, 2-opt и межмаршрутными обменами.
lkh-wrapper-v1...v3	Ранние LKH-подобные версии; используются для проверки эффекта перехода от простой обертки к candidate-based логике.
lkh-wrapper-v4...v7	Версии для крупных инстансов с геометрическими кандидатами, кэшированием расстояний, бюджетами времени и ускоренными процедурами улучшения.
lkh-wrapper-v12	MINSUM-ориентированная версия: строит быстрое первое решение, использует savings/cluster-aware идеи и принимает межмаршрутные изменения по суммарной длине маршрутов.
OR-Tools GLS	Внешний эвристический ориентир на малых и средних инстансах.
TSP-transform + LKH	Внешний baseline: строит TSP-тур по клиентам и затем разбивает его на маршруты динамическим программированием.
LKH-3 baseline	Наиболее сильный внешний ориентир для части ручных сравнений.

2.4 Особенности LKH-подобной реализации

Поздние версии `lkh-wrapper` не являются прямым запуском LKH-3 внутри C++-кода. Это собственные эвристические обертки, вдохновленные LKH-подходом и адаптированные под быстрый экспериментальный контур. Их общая логика состоит из нескольких этапов:

1. построение геометрических `candidate sets`: для малых инстансов возможен точный перебор ближайших соседей, для крупных применяется пространственная сетка;
2. построение начальных маршрутов: используется жадная или кластерно-ориентированная процедура, учитывающая расстояние до депо и распределение точек;
3. локальное улучшение маршрутов: 2-opt ограничивается кандидатными ребрами, чтобы не выполнять полный квадратичный просмотр;
4. межмаршрутные изменения: вершины или блоки переносятся между маршрутами, если это уменьшает `MINSUM` или улучшает баланс;
5. соблюдение бюджета времени: крупные циклы регулярно проверяют `deadline` и возвращают лучшее найденное решение.

В `lkh-wrapper-v12` отдельно выделяется бюджет на получение первого решения и бюджет на улучшение. Такая схема важна для прикладного режима: даже если улучшение не успело завершиться, алгоритм должен вернуть валидное решение. В коде также используется кэш расстояний для крупных инстансов, так как хранение полной матрицы расстояний становится чрезмерным по памяти.

Candidate sets → fast seed routes → savings/cluster seed → route sanitizing
→ local ILS → inter-route relocate/swap → final validation

Рис. 2: Упрощенная логика работы поздней LKH-подобной версии

Нужно добавить перед сдачей. Для защиты отчета полезно добавить 2-3 изображения маршрутов: равномерный пример, кластерный пример и пример с выбросами. В репозитории уже есть JSON с маршрутами, поэтому рисунки можно построить отдельным Python-скриптом и вставить как PNG в приложение.

3 МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА

3.1 Семейства тестовых инстансов

Чтобы не проверять алгоритмы только на одном типе геометрии, в проекте используются несколько семейств синтетических данных. Такой дизайн лучше отражает разные сценарии маршрутизации и снижает риск, что алгоритм случайно хорошо работает только на равномерных точках.

Семейство	Смысл проверки	Депо
uniform	Равномерное распределение точек. Базовый сценарий без выраженной структуры.	центр
clustered-center	Несколько естественных групп клиентов. Проверяется умение алгоритма сохранять компактные группы.	центр
clustered-offset-depot	Кластерная структура при смещенном депо. Сценарий сложнее, потому что ближайшие кластеры и баланс маршрутов конфликтуют.	край
mixed-outliers / mixed	Основная масса точек находится в группах, но есть удаленные выбросы. Проверяется устойчивость к длинным хвостам.	центр
outliers	Усиленный сценарий с удаленными точками. Важен для анализа межмаршрутного перераспределения.	центр
high-m-stress	Повышенная нагрузка по числу агентов и маршрутов. Проверяется устойчивость распределения вершин между маршрутами.	центр

В ранних экспериментах использовалась сетка $n \in \{20, 50, 100, 200\}$, $m \in \{2, 3, 5\}$ и 10 повторов. Позднее она была расширена семействами инстансов и отдельными крупными задачами до $n = 100000$.

3.2 Метрики

Основная метрика качества --- значение MINSUM. Для сравнения с внешним ориентиром используется относительное отклонение:

$$\text{gap}(A, B) = 100 \cdot \frac{F(A) - F(B)}{F(B)}, \quad (4)$$

где $F(A)$ --- MINSUM исследуемого алгоритма, а $F(B)$ --- MINSUM базового или reference-решения. Если gap положителен, исследуемый алгоритм хуже ориентира по MINSUM; если отрицателен, он лучше.

Дополнительно учитываются:

- среднее время работы;
- число валидных запусков;
- число запусков, где алгоритм превзошел reference;
- баланс маршрутов, измеряемый как отношение максимальной длины маршрута к средней.

В работе важно не смешивать разные типы сравнения. OR-Tools с лимитом 5 секунд не является точным оптимумом. Это внешний эвристический ориентир. Поэтому выводы формулируются не как «алгоритм дал оптимальное решение», а как «алгоритм имеет такое-то отклонение от выбранного reference при таких-то условиях».

3.3 Валидация результатов

Каждый результат должен удовлетворять трем условиям: все маршруты начинаются и заканчиваются в депо, все клиентские вершины посещены ровно один раз, значение MINSUM пересчитано независимой функцией по сохраненным маршрутам. Такой подход особенно важен при сравнении нескольких поколений алгоритмов: ускоренная версия может выглядеть привлекательной по времени, но если она возвращает невалидные маршруты, ее нельзя учитывать в агрегатах качества.

Нужно добавить перед сдачей. В итоговом архиве к сдаче стоит приложить короткий README с командами воспроизведения: генерация инстансов, запуск benchmark, пересчет отчетов. Это усилит раздел о достоверности результатов.

4 РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ

4.1 Базовое сравнение на равномерных инстансах

Первый набор экспериментов сравнивал rand+nn, 2opt+greed, grasp и lkh-wrapper-v2 на равномерных инстансах малой и средней размерности. По агрегированным результатам lkh-wrapper-v2 дал лучшее

среднее значение MINSUM среди внутренних алгоритмов, при этом оставался значительно быстрее GRASP.

Таблица 4: Агрегированное сравнение внутренних алгоритмов на равномерных инстансах

Алгоритм	Запуски	Средний MINSUM	Время, с	Валидные
rand+nn	10	1571.74	0.0005	10
2opt+greed	10	1042.94	0.0007	10
grasp	10	866.91	1.0469	10
lkh-wrapper-v2	10	821.33	0.0181	10

Результат подтверждает ожидаемую иерархию методов. Чистый randomized nearest neighbor работает почти мгновенно, но имеет худшее качество. Добавление 2-opt резко улучшает результат. GRASP дает еще более качественные маршруты, но платит за это временем. lkh-wrapper-v2 оказывается наиболее удачным компромиссом в этом наборе: его средний MINSUM ниже, чем у GRASP, а среднее время меньше примерно на два порядка.

4.2 Сравнение с OR-Tools

Для внешнего ориентира использовался OR-Tools Routing с Guided Local Search и лимитом 5 секунд. В таком сравнении все внутренние методы в среднем уступают OR-Tools по MINSUM, но разрыв существенно различается.

Таблица 5: Средний gap к OR-Tools на равномерных инстансах

Алгоритм	Gap, %	Время, с	Лучше ref.
rand+nn	114.73	0.0005	0.00
2opt+greed	47.41	0.0007	0.00
grasp	20.77	1.0469	0.00
lkh-wrapper-v2	14.48	0.0181	0.08

Эти данные не означают, что OR-Tools всегда лучше при любом временном бюджете. Его лимит в данном сравнении составляет 5 секунд, тогда как часть внутренних алгоритмов работает за миллисекунды. Однако таблица по-

лезна для оценки качества: она показывает, насколько далеко находятся быстрые эвристики от сильного внешнего решения.

4.3 Расширенный benchmark по семействам инстансов

После добавления нескольких семейств инстансов картина стала более содержательной. На расширенном наборе `lkh-wrapper-v2` и `lkh-wrapper-v3` заметно превосходят ранние версии, GRASP и простые baseline-алгоритмы по среднему MINSUM. При этом `lkh-wrapper-v3` обычно быстрее `v2`, но по качеству немного уступает или находится рядом.

Таблица 6: Средние значения по расширенному multi-family benchmark

Алгоритм	Средний MINSUM	Среднее время, с
rand+nn	1456.23	0.0007
2opt+greed	999.30	0.0008
grasp	820.59	1.5138
lkh-wrapper-v1	875.99	0.0395
lkh-wrapper-v2	677.61	0.0198
lkh-wrapper-v3	683.44	0.0149

В разрезе семейств видно, что преимущество LKH-подобных версий сохраняется не только на равномерных точках. Особенно сильный эффект наблюдается на кластерных данных: `lkh-wrapper-v2` снижает средний MINSUM относительно GRASP и ранних LKH-оберток, что согласуется с идеей использования геометрической структуры и более направленного локального поиска.

Таблица 7: Средний MINSUM по семействам для ключевых алгоритмов

Семейство	2opt+greed	grasp	lkh-v2	lkh-v3
clustered-center	811.99	663.14	536.62	539.58
clustered-offset-depot	959.50	817.75	538.31	550.34
mixed-outliers	1072.43	863.14	765.89	771.07
uniform	1153.26	938.34	869.62	872.79

4.4 Эволюция LKH-подобных версий

Сравнение `lkh-wrapper-v1`, `v2` и `v3` показывает, что переход от начальной обертки к версиям с более аккуратной candidate-based логикой дал существенный прирост качества. Средний gap к OR-Tools снизился примерно с 29.63% у `v1` до 14.48% у `v2`. Версия `v3` почти повторила качество `v2`, но работала быстрее.

Таблица 8: Сравнение ранних LKH-подобных версий с OR-Tools

Версия	Gap к OR-Tools, %	Время, с	Валидные
<code>lkh-wrapper-v1</code>	29.63	0.0297	10
<code>lkh-wrapper-v2</code>	14.48	0.0186	10
<code>lkh-wrapper-v3</code>	14.61	0.0102	10

Это важный результат для курсовой работы: он показывает не просто наличие нескольких запусков, а инженерную динамику. В проекте есть измеримый эффект от изменения алгоритма, а не только разовое сравнение готовых библиотек.

4.5 Крупные инстансы и проблема масштабирования

На крупных инстансах размером 10000--100000 вершин основная сложность смещается от качества локальных перестановок к способности алгоритма вообще успеть построить валидное решение и улучшить его в пределах бюджета. Эксперименты с версиями `v4`, `v5`, `v6` показывают, что ускорение может сопровождаться потерей качества или валидности.

Таблица 9: Версии `v4--v6` на крупных uniform/mixed инстансах с бюджетом около 30 секунд

Версия	MINSUM по валидным	Время, с	Валидность
<code>lkh-wrapper-v4</code>	$2.38 \cdot 10^8$	42.87	1.00
<code>lkh-wrapper-v5</code>	$2.63 \cdot 10^8$	30.09	1.00
<code>lkh-wrapper-v6</code>	$3.37 \cdot 10^5$	28.34	0.25

Среднее значение `v6` в этой таблице нельзя трактовать как преимущество качества, потому что валидными оказались только отдельные запуски. Более корректный вывод заключается в другом: версия `v6` показала нестабильность,

а v4 и v5 возвращали валидные решения, но v5 обычно была быстрее ценой ухудшения MINSUM. Следовательно, для итогового алгоритма требуется одновременно контролировать валидность, time-budget и качество.

4.6 Сравнение v12 с внешним LKH-3 baseline

Поздняя версия lkh-wrapper-v12 была ориентирована на MINSUM и быстрый возврат валидного решения. Ручные сравнения с lkh3-baseline показывают отчетливый компромисс: внешний LKH-3 дает более короткие маршруты, но собственная версия работает существенно быстрее и часто строит более сбалансированные маршруты.

Таблица 10: Сравнение lkh-wrapper-v12 с lkh3-baseline на n=5000, m=5

Инстанс	LKH-3	v12	v12, с	Ускорение v12
uniform	127094.46	159615.23	4.80	7.64
mixed	62667.81	75288.82	4.83	5.81
outliers	60255.93	71603.77	4.80	5.72
clustered-center	27837.74	37140.17	4.73	5.98
high-m-stress	61312.38	84752.47	4.83	5.86

В среднем на этих пяти инстансах LKH-3 лучше по MINSUM примерно на 21%, а v12 быстрее примерно в 6.2 раза. Это не подтверждает сильную версию гипотезы о превосходстве собственного LKH-подобного алгоритма над LKH-3 по качеству, но подтверждает более практичный вывод: при жестком бюджете времени собственная версия может давать валидное решение быстро, а дальнейшая работа должна быть направлена на сокращение разрыва по objective.

Таблица 11: Баланс маршрутов в сравнении LKH-3 и v12 на n=5000

Инстанс	Imbalance LKH-3	Imbalance v12
uniform	4.99	1.01
mixed	3.85	2.74
outliers	3.88	2.64
clustered-center	1.73	1.04
high-m-stress	4.99	1.11

Баланс маршрутов не является основной целью при MINSUM, однако таблица показывает интересный побочный эффект. `v12` чаще строит более равномерные маршруты, тогда как LKH-3 в режиме MINSUM может концентрировать больше нагрузки в отдельных маршрутах. Если прикладная задача требует одновременно низкий MINSUM и ограничение на максимальную длину маршрута, эту особенность можно использовать как основу для следующей версии алгоритма.

4.7 Интерпретация результатов

Выполненные эксперименты позволяют сформулировать несколько выводов.

Во-первых, простые эвристики полезны как baseline, но недостаточны для качественного решения mTSP. `rand+nn` и `2opt+greed` работают очень быстро, однако дают заметно худшую MINSUM.

Во-вторых, использование candidate-based локального поиска оправдано. Переход от ранних LKH-оберток к `v2/v3` почти вдвое снижает средний gap к OR-Tools на равномерных инстансах и дает устойчивый выигрыш на разных семействах данных.

В-третьих, масштабирование нельзя оценивать только по времени. Версия может быть быстрой, но невалидной или слишком сильно проигрывать по objective. Поэтому в отчете отдельно фиксируются valid runs и не делается вывод о качестве по строкам, где валидность низкая.

В-четвертых, внешний LKH-3 пока остается более сильным ориентиром по качеству. Собственная `v12` версия выигрывает по скорости и балансу, но проигрывает по MINSUM. Это честный и важный результат: он показывает, что проект находится не в состоянии «готовый промышленный решатель лучше всех», а в состоянии исследовательского прототипа с понятным направлением развития.

Нужно добавить перед сдачей. Если останется время, стоит добавить парный статистический тест или хотя бы таблицу wins/losses/ties по одинаковым инстансам для пар `lkh-wrapper-v2` vs `grasp`, `v12` vs `lkh3-baseline`. В материалах уже есть per-instance CSV/JSON, поэтому это можно сделать без новых запусков.

5 ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ

Достоверность результатов обеспечивается несколькими мерами. Все алгоритмы запускаются через единый runner, что снижает риск различий в формате входа и выхода. Для каждого решения проводится независимая проверка посещения вершин и пересчет MINSUM. Результаты сохраняются в CSV/JSON, что позволяет повторно агрегировать данные без ручного переноса чисел. Сравнения выполняются на одних и тех же инстансах, поэтому различия между алгоритмами связаны с их поведением, а не с различием входных данных.

При этом работа имеет ограничения. Во-первых, часть экспериментов выполнена на синтетических данных. Это правильно для контролируемого исследования, но перед прикладным использованием алгоритмы нужно проверить на реальных географических наборах. Во-вторых, не для всех крупных запусков известна полная аппаратная конфигурация. В-третьих, некоторые сравнения имеют разный временной бюджет: OR-Tools запускался с лимитом 5 секунд, а собственные алгоритмы в ранних экспериментах часто работали значительно меньше. Поэтому выводы о «лучшем» алгоритме делаются отдельно для качества и отдельно для времени.

Еще одно ограничение связано с целью MINSUM. Она минимизирует суммарную длину маршрутов, но не гарантирует справедливого распределения нагрузки между агентами. В результатах видно, что LKH-3 может давать лучший MINSUM, но менее сбалансированные маршруты. Если практическая задача требует ограничить максимальную длину маршрута, постановку нужно расширить до MINMAX или многокритериальной цели.

Основной технический риск проекта --- рост времени локального поиска на больших инстансах. Для его снижения используются candidate sets, пространственная сетка, кэш расстояний и проверка бюджета времени. Второй риск --- невалидные решения в ускоренных версиях. Он снижается отдельной фазой sanitize/complete routes и независимой валидацией.

ЗАКЛЮЧЕНИЕ

В ходе работы была рассмотрена задача множественного коммивояжера с одним депо и целевой функцией MINSUM. Были изучены классические и современные подходы к TSP/mTSP, включая локальный поиск, GRASP, LKH, LKH-3, candidate sets и идеи масштабирования для крупных маршрутизацион-

ных задач. На основе этого был реализован экспериментальный контур, позволяющий генерировать инстансы, запускать несколько классов алгоритмов, проверять валидность маршрутов и агрегировать результаты.

Эксперименты показали, что LKH-подобные методы существенно превосходят простые baseline-алгоритмы по качеству решения. На равномерных инстансах `lkh-wrapper-v2` получил средний gap к OR-Tools около 14.48%, тогда как GRASP имел около 20.77%, `2opt+greedy` --- 47.41%, а `rand+nn` --- 114.73%. На расширенном наборе семейств `lkh-wrapper-v2` и `v3` также дали лучшие средние значения MINSUM среди внутренних алгоритмов.

При переходе к крупным задачам была выявлена ключевая инженерная проблема: ускорение алгоритма должно контролироваться вместе с валидностью и качеством. Версии `v4/v5/v6` показали разные стороны этого компромисса: `v5` быстрее `v4`, но обычно хуже по MINSUM, а `v6` не обеспечила достаточную валидность. Поздняя MINSUM-ориентированная версия `v12` в ручных сравнениях с LKH-3 оказалась быстрее примерно в 6 раз на инстансах $n = 5000$, но уступила по суммарной длине маршрутов примерно на 21%. При этом `v12` часто давала более сбалансированные маршруты, что может быть полезно для дальнейшей многокритериальной постановки.

Таким образом, цель работы в исследовательском смысле достигнута: разработан и проанализирован набор эвристических методов для mTSP, построен воспроизводимый экспериментальный контур, получены сравнения с базовыми и внешними алгоритмами, выявлены сильные и слабые стороны текущих решений. Рабочая гипотеза подтверждена частично. LKH-подобные эвристики действительно лучше простых конструктивных методов и GRASP на рассмотренных наборах, но пока не превосходят внешний LKH-3 по качеству MINSUM. Дальнейшее развитие проекта целесообразно направить на совмещение скорости `v12` с более сильной процедурой улучшения, добавление статистического анализа и проверку на реальных маршрутизационных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lin S., Kernighan B. W. An Effective Heuristic Algorithm for the Traveling-Salesman Problem // Operations Research. 1973. Vol. 21, No. 2. P. 498--516.
2. Helsgaun K. An Effective Implementation of the Lin--Kernighan Traveling Salesman Heuristic // European Journal of Operational Research. 2000. Vol. 126, No. 1. P. 106--130.
3. Helsgaun K. An Extension of the Lin--Kernighan--Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Roskilde University, 2017.
4. Bektaş T. The Multiple Traveling Salesman Problem: An Overview of Formulations and Solution Procedures // Omega. 2006. Vol. 34, No. 3. P. 209--219.
5. Jonker R., Volgenant T. Technical Note --- An Improved Transformation of the Symmetric Multiple Traveling Salesman Problem // Operations Research. 1988. Vol. 36, No. 1. P. 163--167.
6. Croes G. A. A Method for Solving Traveling-Salesman Problems // Operations Research. 1958. Vol. 6, No. 6. P. 791--812.
7. Taillard É. D., Helsgaun K. POPMUSIC for the Travelling Salesman Problem // European Journal of Operational Research. 2019. Vol. 272, No. 2. P. 420--429.
8. Accorsi L., Vigo D. Routing One Million Customers in a Handful of Minutes. 2023. arXiv:2306.14205.
9. Feo T. A., Resende M. G. C. Greedy Randomized Adaptive Search Procedures // Journal of Global Optimization. 1995. Vol. 6, No. 2. P. 109--133.
10. Zheng J., Hong Y., Xu W., Li W., Chen Y. An Effective Iterated Two-stage Heuristic Algorithm for the Multiple Traveling Salesmen Problem. 2022. arXiv:2201.09424.
11. Bao X., Wang G., Xu L., Wang Z. Solving the Min-Max Clustered Traveling Salesmen Problem Based on Genetic Algorithm // Biomimetics. 2023. Vol. 8, No. 238.
12. Russell R. A. An Effective Heuristic for the M-Tour Traveling Salesman Problem with Some Side Conditions // Operations Research. 1977. Vol. 25, No. 3. P. 517--524.
13. Reinelt G. TSPLIB --- A Traveling Salesman Problem Library // ORSA Journal

- on Computing. 1991. Vol. 3, No. 4. P. 376--384.
14. Johnson D. S. A Theoretician's Guide to the Experimental Analysis of Algorithms // Data Structures, Near Neighbor Searches, and Methodology. 1999. P. 215--250.
 15. Hooker J. N. Testing Heuristics: We Have It All Wrong // Journal of Heuristics. 1995. Vol. 1, No. 1. P. 33--42.
 16. Google OR-Tools. Routing Library Documentation. Электронный ресурс: <https://developers.google.com/optimization/routing>.
 17. Dantzig G. B., Ramser J. H. The Truck Dispatching Problem // Management Science. 1959. Vol. 6, No. 1. P. 80--91.
 18. Toth P., Vigo D. Vehicle Routing: Problems, Methods, and Applications. SIAM, 2014.
 19. Материалы репозитория `mtsp-routing-optimization`: исходный код решателей, скрипты экспериментов и CSV/JSON-артефакты результатов. Локальный архив проекта, 2026.

ПРИЛОЖЕНИЕ А. ЧТО НУЖНО ДОБАВИТЬ В ФИНАЛЬНУЮ ВЕРСИЮ ПЕРЕД СДАЧЕЙ

В основной текст уже включена структура полноценного итогового отчета: реферат, определения, введение, обзор предметной области, постановка задачи, реализация, методика эксперимента, результаты, ограничения, заключение и список источников. Ниже перечислены элементы, которые желательно добавить после уточнения данных.

1. Точный объем отчета: число страниц, таблиц, рисунков, источников и приложений.
2. Полная аппаратная конфигурация экспериментов: CPU, RAM, ОС, компилятор, режим сборки, число потоков.
3. Скриншоты или PNG-визуализации маршрутов для 2--3 характерных инстансов.
4. Таблица wins/losses/ties по одинаковым инстансам для ключевых пар алгоритмов.
5. Полная таблица per-instance результатов в приложении или отдельном CSV-архиве.
6. Команды воспроизведения экспериментов и ссылка на репозиторий/архив исходного кода.
7. Финальная сверка всех чисел после последнего запуска benchmark, если будут добавлены новые эксперименты.

ПРИЛОЖЕНИЕ Б. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБЛИЦЫ ЭКСПЕРИМЕНТОВ

Полную таблицу не следует перегружать в основной части отчета. Ее лучше вынести в приложение или приложить отдельным CSV. Рекомендуемый набор столбцов:

Столбец	Содержание
instance_family	Семейство инстанса: uniform, clustered-center, mixed, outliers и т.д.
instance	Имя конкретного файла с тестом.
node_count	Число вершин.
salesman_count	Число агентов.
solver	Название алгоритма.
valid	Результат проверки корректности маршрутов.
objective	Пересчитанное значение MINSUM.
time_seconds	Время выполнения.
reference_objective	Значение внешнего ориентира, если он использовался.
relative_gap_percent	Относительное отклонение от reference.
imbalance	Отношение максимальной длины маршрута к средней длине маршрута.
seed	Seed запуска, если алгоритм стохастический.